

What is Pandas?

Pandas is an open-source Python library used for data manipulation, analysis, and cleaning. It is built on top of NumPy.

Installing Pandas

```
pip install pandas
```

Importing Pandas

```
import pandas as pd
```

Core Data Structures

1. Series – One-dimensional labeled array
2. DataFrame – Two-dimensional labeled data structure (like a table)

Creating a Series

```
import pandas as pd

# From list
s = pd.Series([10, 20, 30])
# With custom index
s = pd.Series([10, 20, 30], index=['a', 'b', 'c'])
```

Creating a DataFrame

```
# From dictionary
data = {
    'Product': ['Laptop', 'Tablet', 'Mobile'],
    'Price': [60000, 20000, 15000]
}
df = pd.DataFrame(data)
```

Basic DataFrame Operations

```
df.head()      # First 5 rows
df.tail(3)     # Last 3 rows
```

```
df.shape      # Rows and columns
df.info()     # Info summary
df.describe() # Statistical summary
df.columns    # List of columns
df.dtypes     # Data types of each column
```

Accessing Data

```
df['Price']      # Access single column
df[['Product', 'Price']] # Access multiple columns
df.loc[0]        # Access row by index label
df.iloc[1]       # Access row by integer location
```

Modifying Data

```
df['Discount'] = [5, 10, 15] # Add new column
df['NetPrice'] = df['Price'] - df['Discount']
df.drop('Discount', axis=1)  # Drop column (not in-place)
df.drop(0, axis=0)          # Drop row (not in-place)
```

Data Selection & Filtering

```
# Filter rows with condition
df[df['Price'] > 20000]

# Multiple conditions
df[(df['Price'] > 15000) & (df['Product'] != 'Tablet')]

# Using query
df.query("Price > 15000 and Product != 'Tablet'")
```

Sorting Data

```
# Sort by column values
df.sort_values(by='Price')

# Sort descending
df.sort_values(by='Price', ascending=False)

# Sort by multiple columns
df.sort_values(by=['Price', 'Product'])
```

Renaming Columns & Index

```
# Rename columns
df.rename(columns={'Price': 'Cost'})

# Rename index
df.rename(index={0: 'A', 1: 'B'})

# In-place rename
df.rename(columns={'Price': 'Cost'}, inplace=True)
```

Handling Missing Data

```
# Detect missing values
df.isnull()

# Count missing values
df.isnull().sum()

# Fill missing values
df.fillna(0)

# Fill with method
df.fillna(method='ffill') # forward fill
df.fillna(method='bfill') # backward fill

# Drop missing values
df.dropna()
```

Changing Data Types

```
# Convert data type
df['Price'] = df['Price'].astype(float)

# Convert to datetime
df['Date'] = pd.to_datetime(df['Date'])
```

Basic Statistics

```
df['Price'].mean()
df['Price'].median()
df['Price'].mode()
df['Price'].sum()
df['Price'].min()
df['Price'].max()
df['Price'].std()
df['Price'].var()
```

Grouping and Aggregation

```
# Group by a single column
df.groupby('Category').sum()

# Group by multiple columns
df.groupby(['Category', 'SubCategory']).mean()

# Aggregation with custom functions
df.groupby('Category').agg({
    'Price': 'sum',
    'Discount': 'mean'
})
```

Common Aggregation Functions

```
.sum()
.mean()
.count()
.min()
.max()
.median()
.std()
```

Reset Index After Grouping

```
grouped = df.groupby('Category').sum().reset_index()
```

Filtering Groups

```
# Get group object
group_obj = df.groupby('Category')

# Access a single group
group_obj.get_group('Electronics')

# Filter groups with filter function
df.groupby('Category').filter(lambda x: x['Price'].mean() > 20000)
```

Merging DataFrames (SQL-like Joins)

```
# Merge on common column
pd.merge(df1, df2, on='ProductID')

# Left join
pd.merge(df1, df2, on='ProductID', how='left')

# Inner join
pd.merge(df1, df2, on='ProductID', how='inner')

# Outer join
pd.merge(df1, df2, on='ProductID', how='outer')
```

Concatenation

```
# Concatenate rows (vertical)
pd.concat([df1, df2])

# Concatenate columns (horizontal)
pd.concat([df1, df2], axis=1)
```

Appending Rows

```
df.append(new_row, ignore_index=True)
```

DateTime Handling

```
# Convert column to datetime
df['OrderDate'] = pd.to_datetime(df['OrderDate'])

# Extract parts of date
df['Year'] = df['OrderDate'].dt.year
df['Month'] = df['OrderDate'].dt.month
df['Day'] = df['OrderDate'].dt.day
df['Weekday'] = df['OrderDate'].dt.day_name()

# Filter by date
df[df['OrderDate'] >= '2024-01-01']
```

Pivot Table

```
# Basic pivot table
pd.pivot_table(df, index='Category', values='Price', aggfunc='sum')

# Multiple agg functions
pd.pivot_table(df, index='Category', columns='SubCategory',
values='Price', aggfunc='mean')

# Add totals
pd.pivot_table(df, index='Category', values='Price', aggfunc=['sum',
'mean'], margins=True)
```

Apply Function

```
# Apply custom function to column
df['DiscountedPrice'] = df['Price'].apply(lambda x: x * 0.9)

# Apply to row or column (axis=1 for row)
df.apply(lambda row: row['Price'] - row['Discount'], axis=1)
```

Map Function

```
# Map values in column
df['CategoryCode'] = df['Category'].map({'Electronics': 1, 'Fashion': 2})
```

Replace Values

```
df['Product'].replace('Laptop', 'Notebook', inplace=True)

# Replace multiple
df['Category'].replace({'Electronics': 'E', 'Fashion': 'F'},
inplace=True)
```

Cut & Binning

```
# Bin continuous data into ranges
df['PriceRange'] = pd.cut(df['Price'], bins=[0, 10000, 30000, 60000],
labels=['Low', 'Medium', 'High'])
```

Categorical Data

```
# Convert to category dtype
df['Category'] = df['Category'].astype('category')

# Check categories
df['Category'].cat.categories

# Rename categories
df['Category'] = df['Category'].cat.rename_categories({'E':
'Electronics'})
```

Multindexing (Hierarchical Indexing)

```
# Create multi-index dataframe
df_multi = df.set_index(['Category', 'SubCategory'])

# Accessing data
df_multi.loc[('Electronics', 'Laptop')]

# Reset to normal index
```

```
df_multi.reset_index(inplace=True)
```

Window Functions (Rolling, Expanding)

```
# Rolling window (moving average)
df['RollingAvg'] = df['Price'].rolling(window=3).mean()

# Expanding window (cumulative)
df['CumulativeAvg'] = df['Price'].expanding().mean()

# Cumulative sum
df['Cumsum'] = df['Price'].cumsum()
```

Working with CSV Files

```
# Read CSV
df = pd.read_csv('data.csv')

# Write to CSV
df.to_csv('output.csv', index=False)

# Preview data
df.head()
```

Working with Excel Files

```
# Read Excel
df = pd.read_excel('data.xlsx', sheet_name='Sheet1')

# Write to Excel
df.to_excel('output.xlsx', sheet_name='Report', index=False)
```

Performance Tips

```
# Use specific dtypes while reading
df = pd.read_csv('data.csv', dtype={'ProductID': 'int32', 'Price': 'float32'})

# Use categorical for repetitive strings
df['Category'] = df['Category'].astype('category')
```

```
# Filter only needed columns while reading
df = pd.read_csv('data.csv', usecols=['ProductID', 'Price', 'Category'])

# Use vectorized operations instead of loops
df['Discounted'] = df['Price'] * 0.9 # instead of for-loop
```

Exporting Cleaned Data

```
# Export cleaned DataFrame to CSV or Excel
df_cleaned.to_csv('cleaned_data.csv', index=False)
df_cleaned.to_excel('cleaned_data.xlsx', index=False)
```

Working with JSON Data

```
# Read JSON file
df = pd.read_json('data.json')

# Convert DataFrame to JSON
df.to_json('output.json', orient='records', lines=True)
```

Working with SQL Databases

```
import sqlite3
from sqlalchemy import create_engine

# Create a connection
conn = sqlite3.connect('mydata.db')

# Read from SQL table
df = pd.read_sql('SELECT * FROM products', conn)

# Write DataFrame to SQL
df.to_sql('new_table', conn, if_exists='replace', index=False)

# Using SQLAlchemy for MySQL or PostgreSQL
engine = create_engine('mysql+pymysql://user:password@localhost/dbname')
df.to_sql('products', engine, if_exists='append', index=False)
```

```
from sqlalchemy import create_engine

# Replace with your MySQL credentials and database
engine = create_engine(
    "mysql+pymysql://root:1234@localhost:3306/world"
)

# Export df to MySQL
df.to_sql(
    name="cat_df",
    con=engine,
    if_exists="replace",  # use "append" if you want to add rows
    instead
    index=False
)
```

Time Series in Pandas

```
# Create time-series index
df['Date'] = pd.to_datetime(df['Date'])
df.set_index('Date', inplace=True)

# Resample data
df.resample('M').sum()  # Monthly
df.resample('W').mean() # Weekly average
df.resample('Y').max()  # Yearly max

# Shift data
df['Price_Yesterday'] = df['Price'].shift(1)
```

Visualization using Pandas (Built-in Plotting)
(Requires matplotlib installed)

```
import matplotlib.pyplot as plt

# Line plot
df['Price'].plot()
```

```
# Bar plot
df.groupby('Category')['Price'].mean().plot(kind='bar')

# Pie chart
df['Category'].value_counts().plot.pie(autopct='%1.1f%%')

# Histogram
df['Price'].plot.hist(bins=10)

plt.show()
```



Pandas Interview Questions

1. What are the key differences between a Series and a DataFrame in Pandas?
2. How do you handle missing data in a DataFrame?
3. What is the difference between `loc[]` and `iloc[]`?
4. How can you merge two DataFrames with different keys?
5. Explain the use of `groupby()` and `pivot_table()` with examples.
6. What are vectorized operations in Pandas? Why are they important?
7. How would you filter rows based on multiple conditions in Pandas?
8. Describe how to create and work with MultiIndex DataFrames.
9. How can you optimize performance in large Pandas datasets?
10. What is the difference between `apply()`, `map()`, and `applymap()`?

Real-World Use Case

1. Data Cleaning: Remove duplicates, fix missing data, normalize column names.
2. Exploratory Data Analysis (EDA): Use `describe()`, `value_counts()`, and visual plots.
3. Data Transformation: Use functions like `apply()`, `map()`, `replace()`, `cut()` for mass transformations.
4. Joining Multiple Sources: Merge multiple Excel/CSV/SQL files into one clean DataFrame.
5. Performance: Use chunked reading (`pd.read_csv(..., chunksize=10000)`), memory-optimized dtypes, and categorical data.
6. DateTime: Always parse dates while reading files (`parse_dates`), and set datetime index for time series.
7. Exporting Results: Always save processed data (`to_csv`, `to_excel`) for downstream use.